

UML 2 Use Case Diagram Overview

by [Scott W. Ambler](#), Copyright 2003

The use case diagram in [Figure 1](#) provides an example of an UML use case diagram. Use case diagrams depict:

- **Use cases.** A use case describes a sequence of actions that provide something of measurable value to an actor and is drawn as a horizontal ellipse.
- **Actors.** An actor is a person, organization, or external system that plays a role in one or more interactions with your system. Actors are drawn as stick figures.
- **Associations.** Associations between actors and use cases are indicated in use case diagrams by solid lines. An association exists whenever an actor is involved with an interaction described by a use case. Associations are modeled as lines connecting use cases and actors to one another, with an optional arrowhead on one end of the line. The arrowhead is often used to indicating the direction of the initial invocation of the relationship or to indicate the primary actor within the use case. The arrowheads are typically confused with data flow and as a result I avoid their use.
- **System boundary boxes (optional).** You can draw a rectangle around the use cases, called the system boundary box, to indicate the scope of your system. Anything within the box represents functionality that is in scope and anything outside the box is not. System boundary boxes are rarely used, although on occasion I have used them to identify which use cases will be delivered in each major release of a system (Ambler 2003b). [Figure 2](#) shows how this could be done.
- **Packages (optional).** Packages are UML constructs that enable you to organize model elements (such as use cases) into groups. Packages are depicted as file folders and can be used on any of the UML diagrams, including both use case diagrams and class diagrams. I use packages only when my diagrams become unwieldy, which generally implies they cannot be printed on a single page, to organize a large diagram into smaller ones. [Figure 3](#) depicts how [Figure 1](#) could be reorganized with packages.

In the example depicted in [Figure 1](#) students are enrolling in courses with the potential help of registrars. Professors input the marks students earn on assignments and registrars authorize the distribution of transcripts (report cards) to students. Note how for some use cases there is more than one actor involved. Moreover, note how some associations have arrowheads—any given use case association will have a zero or one arrowhead. The association between *Student* and *Enroll in Seminar* indicates this use case is initially invoked by a student and not by a registrar (the *Registrar* actor is also involved with this use case). Understanding that associations don't

represent flows of information is important; they merely indicate an actor is somehow involved with a use case. Information is flowing back and forth between the actor and the use case, for example, students would need to indicate which seminars they want to enroll in and the system would need to indicate to the students whether they have been enrolled. However, use case diagrams don't model this sort of information. Information flow can be modeled using [UML activity diagrams](#). The line between the *Enroll in Seminar* use case and the *Registrar* actor has no arrowhead, indicating it is not clear how the interaction between the system and registrars start. Perhaps a registrar may notice a student needs help and offers assistance, whereas other times, the student may request help from the registrar, important information that would be documented in the description of the use case. Actors are always involved with at least one use case and are always drawn on the outside edges of a use case diagram.

Figure 1. System use case diagram.

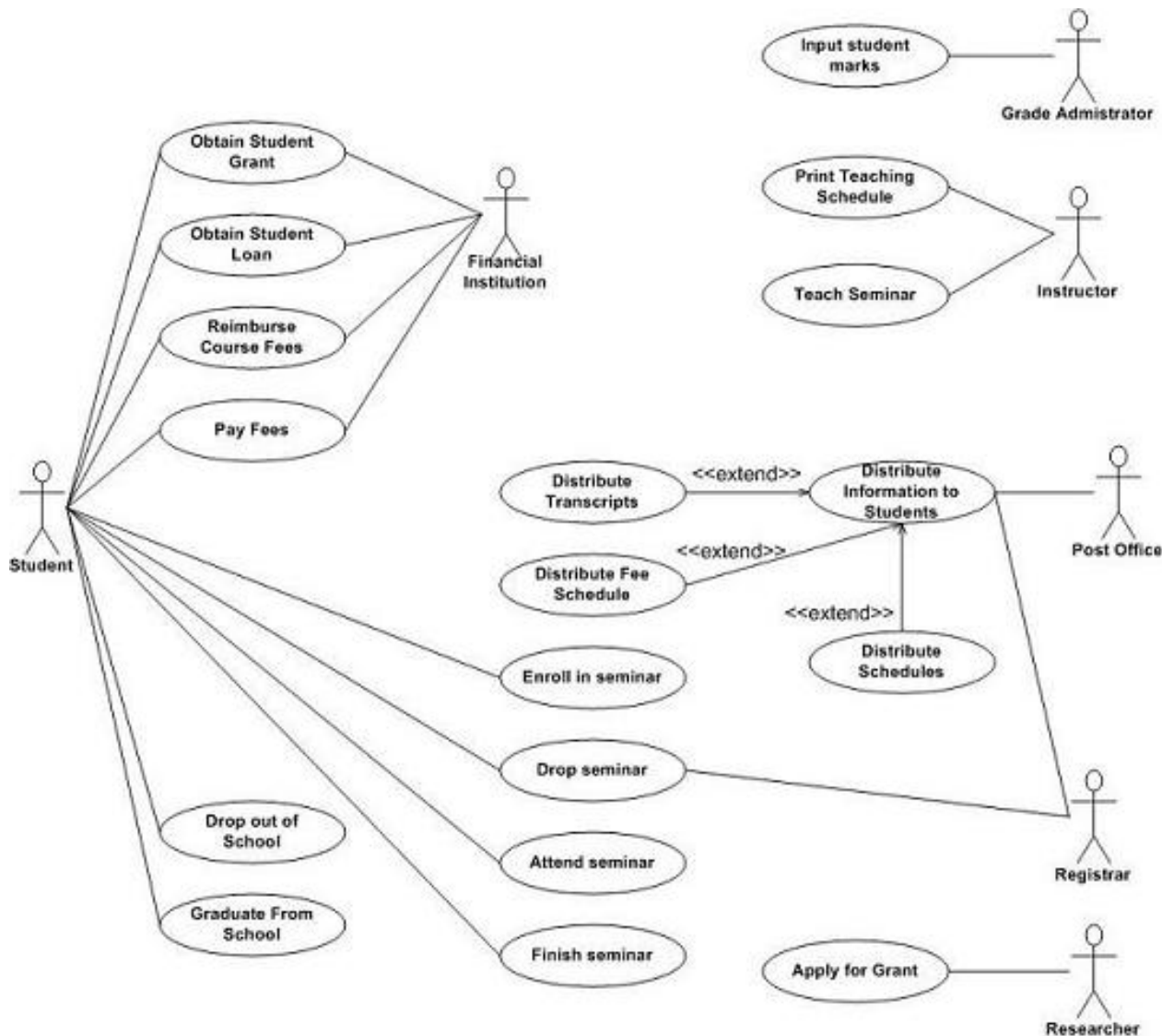


Figure 2. Using System boundary boxes to indicate releases.

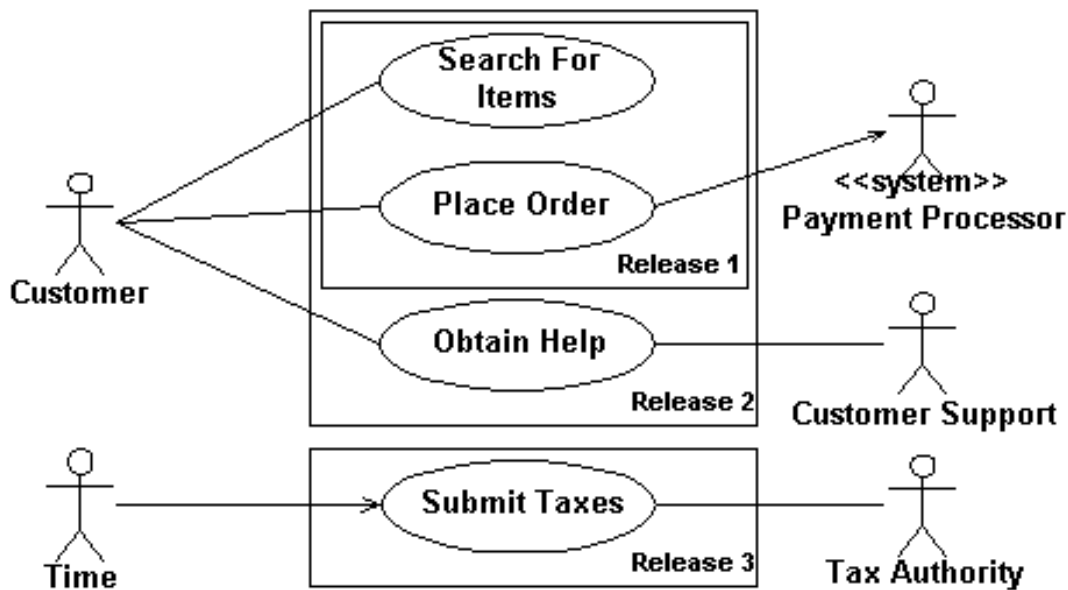
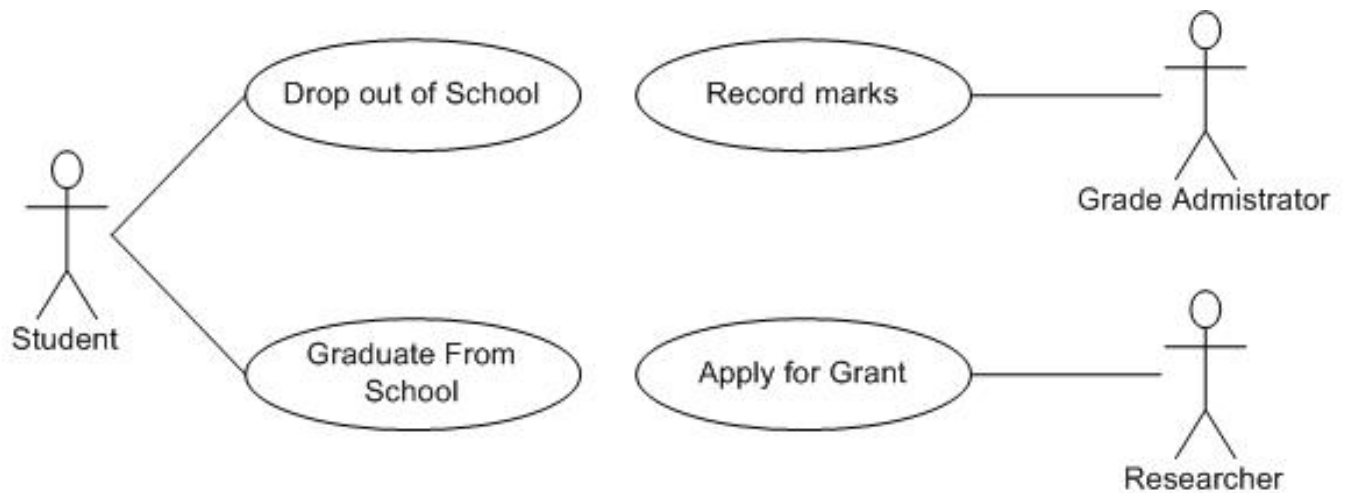


Figure 3. Applying packages to simplify use case diagrams.



Creating Use Case Diagrams

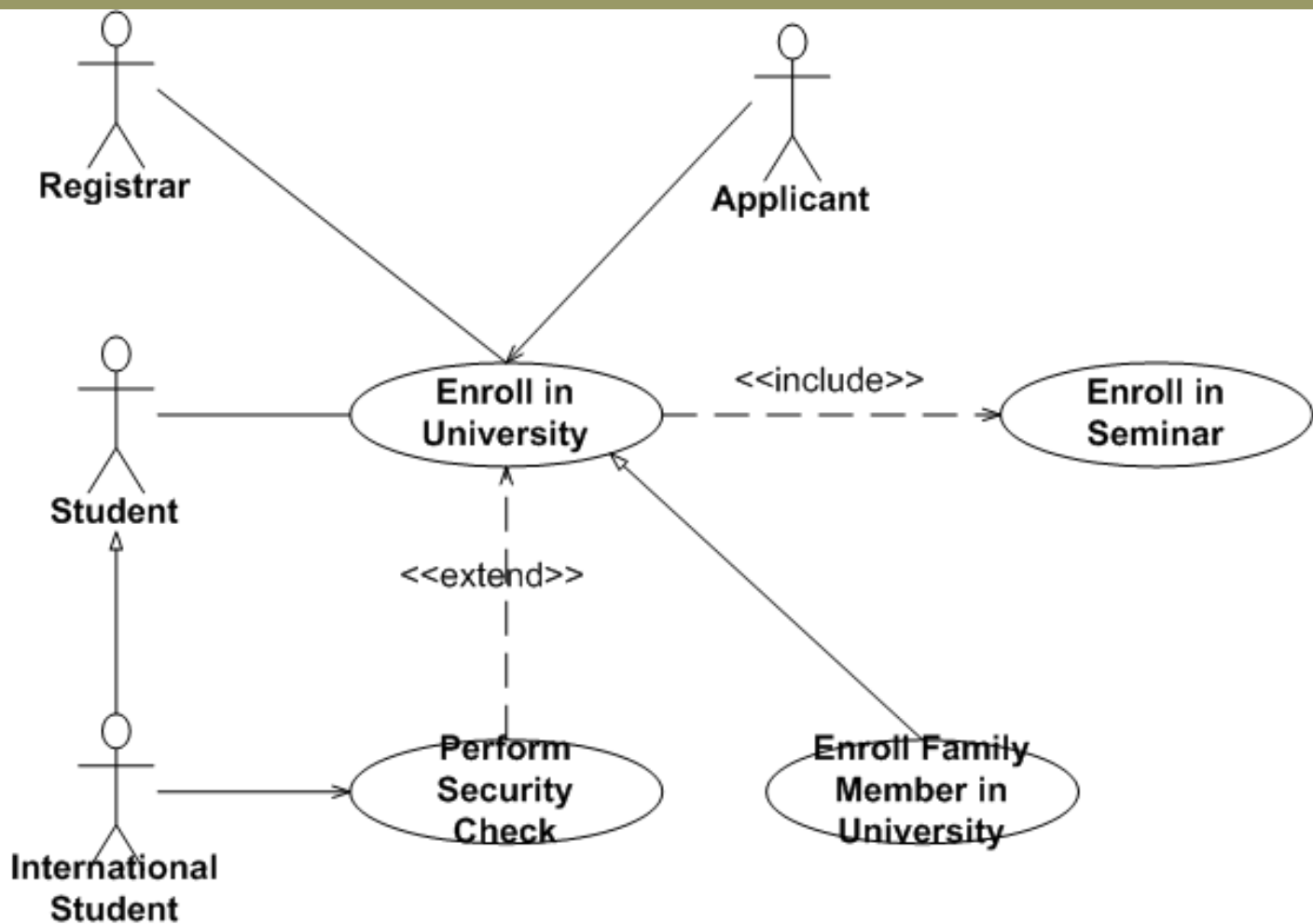
I like to start by identifying as many actors as possible. You should ask how the actors interact with the system to identify an initial set of use cases. Then, on the diagram, you connect the actors with the use cases with which they are involved. If an actor supplies information, initiates the use case, or receives any information as a result of the use case, then there should be an association between them. I generally don't include arrowheads on the association lines because my experience is that people confuse them for indications of information flow, not initial invocation. As I begin to notice similarities between use cases, or between actors, I start modeling the appropriate relationships between them (see the [Reuse Opportunities](#) section).

The preceding paragraph describes my general use case modeling style, an “actors first” approach. Others like to start by identifying one actor and the use cases that they're involved with first and then evolve the model from there. Both approaches work. The important point is that different people take different approaches so you need to be flexible when you're following AM's practice of [Model With Others](#).

Reuse Opportunities

[Figure 4](#) shows the three types of relationships between use cases -- extends, includes, and inheritance -- as well as inheritance between actors. I like to think of extend relationships as the equivalent of a "hardware interrupt" because you don't know when or if the extending use case will be invoked (perhaps a better way to look at this is extending use cases are conditional). Include relationships as the equivalent of a procedure call. Inheritance is applied in the same way as you would on [UML class diagrams](#) -- to model specialization of use cases or actors in this case. The essay [Reuse in Use Case Models](#) describes these relationships in greater detail.

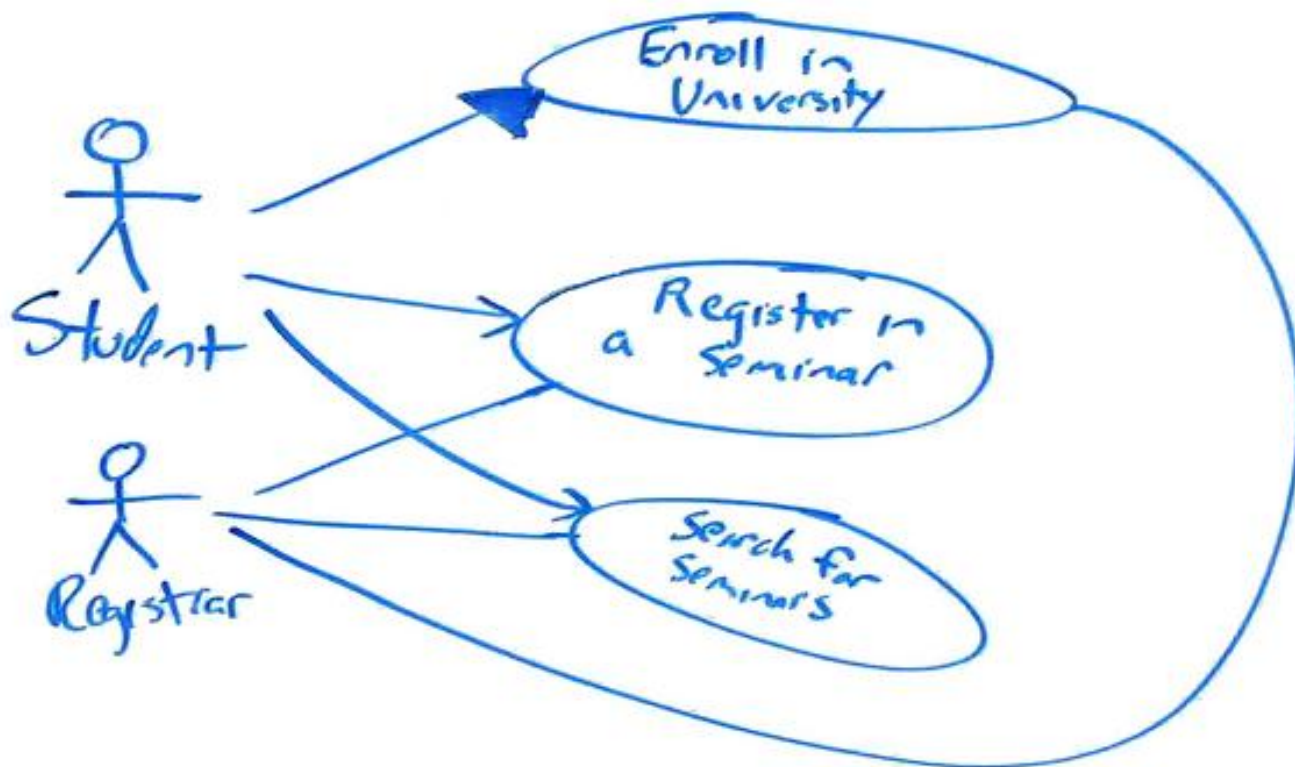
Figure 4. Use case reuse.



Staying Agile

So how can you keep use case modeling agile? First, focus on keeping it as simple as possible. Use simple, flexible tools to model with. I'll typically create use case diagrams on a whiteboard, as you see in [Figure 5](#) which is an example of an initial diagram that I would draw with my project stakeholders. AM tells us that [*Content is More Important Than Representation*](#) so it isn't a big issue that the diagram is hand drawn, it's just barely good enough and that's all that we need. It's also perfectly okay that the diagram isn't complete, there's clearly more to a university than what is depicted, because we can always modify the diagram as we need to.

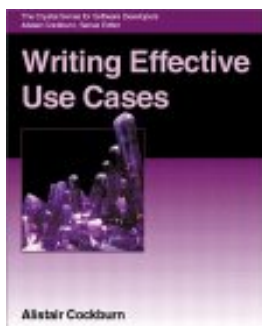
Figure 5. Whiteboard sketch.



In parallel to creating the sketch I would also write a very brief description of each use case, often on a whiteboard as well. The goal is to record just enough information about the use case so that we understand what it is all about. If we need more details we can always add them later either as an [essential/business use case](#) or a [system use case](#).

Notice: This artifact description has been excerpted from [The Object Primer 3rd Edition: Agile Modeling Driven Development with UML 2](#).

Suggested Reading



This is the best book to read if you want to learn how how write use cases effectively.



This is a very good book describing techniques for working with your stakeholders, including use case modeling.

Other artifact
overviews

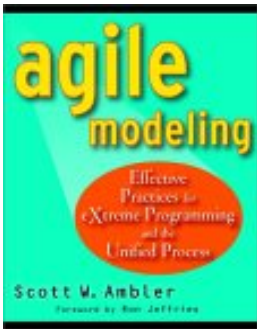
[System Use Cases](#) and [Essential Use Cases](#).

Other Use-Case
Related
Whitepapers

- [UML Use Case Diagram Style Guidelines](#)
- [Reuse in Use Case Models](#)

Additional Reading

To learn more about Agile Modeling (AM), consider the book



- [Agile Modeling Home Page](#)
- [Agile Modeling Essays](#)
- [Agile Data Home Page](#)
- [Artifacts for Agile Modeling: The UML and Beyond](#)
- [Modeling Style Home Page](#)

Let Us Help

[Ronin International, Inc.](#) continues to help numerous organizations to learn about and hopefully

adopt agile techniques and philosophies. We offer both [consulting](#) and [training](#) offerings. In addition we host several sites - [Agile Modeling](#), [Agile Database Techniques](#), [UML Modeling Style Guidelines](#), [Enterprise Unified Process \(EUP\)](#) - that you may find of value.

You might find several of my books to be of interest, including [The Object Primer](#), [Agile Modeling](#), [The Elements of UML Style](#), and [Agile Database Techniques](#).

For more information please contact Michael Vizados at 866-AT-RONIN (U.S. number) or via e-mail (michael.vizados@ronin-intl.com).

A digital counter with a black background and white numbers, displaying the value 16731.

visits since June 19, 2003.

The Official Agile Modeling (AM) Site



[Modeling Artifacts](#)>System Use Case

System Use Case Overview

by [Scott W. Ambler](#), Copyright 2003

A use case is a sequence of actions that provide a measurable value to an actor. Another way to look at it is a use case describes a way in which a real-world actor interacts with the system. In a system use case you include high-level implementation decisions. System use cases can be written in both an [informal manner](#) and a [formal manner](#). Techniques for [identifying use cases](#) are discussed as well as [how to remain agile](#) when writing use cases.

Informal System Use Cases

Let's start by considering the types of use cases that you'll write as part of your initial requirements gathering efforts during "[cycle 0](#)" of your projects. These use cases will either be essential use cases or "informal" system use cases, an example of which is presented in [Figure I-1](#). As you can see the steps are written in very brief, bullet/point-form style. They contain just enough information to get the idea across and no more. It also takes technology issues into account, for example the text "Student inputs her name and address" implies some sort of information system. The reference to the system also implies the same.

Figure I-1. *Enroll in seminar* as an informal system use case (automated solution).

Name: Enroll in Seminar

Identifier: UC 17

Basic Course of Action:

- Student inputs her name and student number
- System verifies the student is eligible to enroll in seminars. If not eligible then student informed and use case ends.
- System displays list of available seminars.
- Student chooses a seminar or decides not to enroll at all.
- System validates the student is eligible to enroll in the chosen seminar. If not eligible student is asked to choose another.
- System validates the seminar fits student's schedule.
- System calculates and displays fees
- Student verifies the cost and either indicates she wants to enroll or not.
- System enrolls the student in the seminar and bills them for it.
- The system prints enrollment receipt.

[Figure I-2](#) presents an alternate version, this time as a manual process involving a registrar (a person) instead of an automated system. Choosing a manual process over a software-based one is still a technical architecture decision, in this case a low-tech architectural decision. The differences between the two versions illuminates how system use cases are not analysis and arguably even design artifacts, not requirements artifacts.

Figure I-2. Enroll in Seminar as an informal use case (manual solution).

Name: Enroll in Seminar

Identifier: UC 17

Basic Course of Action:

- Student inputs her name and student number
- Registrar verifies the student is eligible to enroll in seminars. If not eligible then student informed and use case ends.
- Registrar asks student which seminar they'd like to enroll in. If they don't know registrar provides student with course catalog if required.
- Student chooses a seminar or decides not to enroll at all.
- Registrar checks student record to see if student has previously passed prerequisite courses. If not eligible student is asked to choose another.
- Registrar validates the seminar fits into student's schedule.
- Registrar calculates fees
- Student verifies the cost and either indicates she wants to enroll or not.
- Registrar enrolls the student in the seminar and bills them for it.
- The registrar writes enrollment receipt.

Formal System Use Cases

[Figure 1](#) presents a formalized version of [Figure I-1](#). This version is much more detailed than the corresponding use case, and is typical of the type of use cases that people will write in documentation-intense environments. Frankly, use cases like this are overkill for many projects yet many project teams are required to write them in this manner (or something similar) because senior management is convinced that they require this level of documentation. My advice is to keep your models as simple as possible and only document them this thoroughly if it adds actual value.

A formalized system use case refers to specific user interface components—such as screens, HTML pages, or reports—something you wouldn't do in an [essential/business use case](#). During analysis, you make decisions regarding what will be built, information reflected in your use cases, and, arguably, even how it will be built (effectively design). Because your use cases refer to user interface

components, and because your user interface is worked on during design, inevitably design issues will creep into your use cases. For example, a design decision is whether your user interface is implemented using browser-based technology, such as HTML pages or graphical user interface (GUI) technology such as Windows. Because your user interface will work differently depending on the implementation technology, the logic of your system use cases, which reflect the flow of your user interface, will also be affected.

Figure 1. Enroll in Seminar as a formal system use case.

Name: Enroll in Seminar

Identifier: UC 17

Description:

Enroll an existing student in a seminar for which she is eligible.

Preconditions:

The Student is registered at the University.

Postconditions:

The Student will be enrolled in the course she wants if she is eligible and room is available.

Basic Course of Action:

1. The use case begins when a student wants to enroll in a seminar.

2. The student inputs her name and student number into the system via *UI23 Security Login Screen*.
3. The system verifies the student is eligible to enroll in seminars at the university according to business rule *BR129 Determine Eligibility to Enroll*. [Alt Course A]
4. The system displays *UI32 Seminar Selection Screen*, which indicates the list of available seminars.
5. The student indicates the seminar in which she wants to enroll. [Alt Course B: The Student Decides Not to Enroll]
6. The system validates the student is eligible to enroll in the seminar according to the business rule *BR130 Determine Student Eligibility to Enroll in a Seminar*. [Alt Course C]
7. The system validates the seminar fits into the existing schedule of the student according to the business rule *BR143 Validate Student Seminar Schedule*.
8. The system calculates the fees for the seminar based on the fee published in the course catalog, applicable student fees, and applicable taxes. Apply business rules *BR 180 Calculate Student Fees* and *BR45 Calculate Taxes for Seminar*.
9. The system displays the fees via *UI33 Display Seminar Fees Screen*.
10. The system asks the student if she still wants to enroll in the seminar.
11. The student indicates she wants to enroll in the seminar.
12. The system enrolls the student in the seminar.
13. The system informs the student the enrollment was successful via *UI88 Seminar Enrollment Summary Screen*.
14. The system bills the student for the seminar, according to business rule *BR100 Bill Student for Seminar*.
15. The system asks the student if she wants a printed statement of the

enrollment.

16. The student indicates she wants a printed statement.
17. The system prints the enrollment statement *UI89 Enrollment Summary Report*.
18. The use case ends when the student takes the printed statement.

Alternate Course A: The Student is Not Eligible to Enroll in Seminars.

- A.3. The registrar determines the student is not eligible to enroll in seminars.
- A.4. The registrar informs the student he is not eligible to enroll.
- A.5. The use case ends.

Alternate Course B: The Student Decides Not to Enroll in an Available Seminar

- B.5. The student views the list of seminars and does not see one in which he wants to enroll.
- B.6. The use case ends.

Alternate Course C: The Student Does Not Have the Prerequisites

- C.6. The registrar determines the student is not eligible to enroll in the seminar he chose.
- C.7. The registrar informs the student he does not have the prerequisites.
- C.8. The registrar informs the student of the prerequisites he needs.

C.9. The use case continues at Step 4 in the basic course of action.

Figure 2. *Enroll in University* as a formal system use case.

Name: Enroll in University

Identifier: UC 19

Description:

Enroll someone in the university.

Preconditions:

- The Registrar is logged into the system.
- The Applicant has already undergone initial checks to verify that they are eligible to enroll.

Postconditions:

- The Applicant will be enrolled in the university as a student if they are eligible.

Basic Course of Action:

1. An applicant wants to enroll in the university.
2. The applicant hands a filled out copy of form *UI13 University Application Form* to the registrar. [Alternate Course A: Forms Not Filled Out]

3. The registrar visually inspects the forms.
4. The registrar determines that the forms have been filled out properly.
[Alternate Course B: Forms Improperly Filled Out].
5. The registrar clicks on the *Create Student* icon.
6. The system displays *UI89 Create Student Screen*.
7. The registrar inputs the name, address, and phone number of the applicant.
[Extension Point: *UC34 Perform Security Check*. Applicable to Step 17]
8. The system determines that the applicant does not already exist within the system according to *BR37 Potential Match Criteria for New Students*. [Alternate Course F: Students Appears to Exist Within The System].
9. The system determines that the applicant is on the eligible applicants list.
[Alternate Course G: Person is Not Eligible to Enroll]
10. The system adds the applicant to its records. The applicant is now considered to be a student.
11. The registrar helps the student to enroll in seminars via the use case *UC 17 Enroll in Seminar*.
12. The system calculates the required initial payment in accordance to *BR16 Calculate Enrollment Fees*.
13. The system displays *UI15 Fee Summary Screen*.
14. The registrar asks the student to pay the initial payment in accordance to *BR19 Fee Payment Options*.
15. The student pays the initial fee. [Alternate Course D: The Student Can't Pay At This Time]
16. The system prints a receipt.
17. The registrar hands the student the receipt.

18. The use case ends.

Alternate Course A: Forms Not Filled Out

A.2. The Applicant asks for a set of forms.

A.3. The Applicant fills out the forms as appropriate.

A.4. The use case continues at step 2 in the basic course of action.

Alternate Course B:

[Figure 2](#) presents a formalized system use case (also called a traditional or concrete use case) for enrolling in the university. Interesting points about it:

- The system use case has many implementation details embedded within it. For example, it references "the term system" indicating a decision has been made to automate many of the mundane aspects of enrollment. The writer of system use cases is analyzing and describing requirements imposed by the problem, intermingled with implicit decisions about what the user interface is going to be like.
- The system use case makes references to screen and reports, for example, *UI23 Security Login Screen* and *UI89 Enrollment Summary Report*. Once again this reflects implementation details, someone has decided the system will be implemented as screens, as opposed to HTML pages perhaps, and printed reports.
- The use case references [business rule](#) definitions—such as *BR129 Determine Eligibility to Enroll*—because business rules reflect essential characteristics of your domain that your system must implement. For very simple systems without very many complex business rules I'll often keep it

simple and document the rule within the use case. Different situations call for different approaches, hence the importance of AM's *Local Adaptation* principle.

- Each use case step reflects one activity and one activity only. Several advantages exist to this approach: the use case becomes easier to test because each statement is easier to understand and to validate; alternate courses are easier to write because it is easier to branch from a statement when it does one thing only.
- Use case steps are written in the active voice. For example, the statement “The registrar informs the student of the fees” is in active voice whereas “The student is informed of the fees by the registrar” is in passive voice. Writing in the active voice leads to succinct sentences.
- I like to end the basic course of action within a use case with a closing statement. This is often something along the lines of “The use case ends” or “The use case ends when . . .”, indicating that the logic for the course of action has been completely defined.
- An alternate course of action is an infrequently used path of logic in a use case. Alternate courses are identified whenever there is an alternate way to work, an exception, or an error condition that must be handled. The use case text references several alternate courses, think of them simply as the use case way of doing if/then logic, one of which is described at the bottom of the use case.

Identifying Use Cases

How do you go about identifying potential use cases? [Constantine and Lockwood \(1999\)](#) suggest one way to identify essential use cases, or simply to identify use cases, is to identify potential services by asking your stakeholders the following questions from the point of view of the actors:

- What are users in this role trying to accomplish?
- To fulfill this role, what do users need to be able to do?
- What are the main tasks of users in this role?

- What information do users in this role need to examine, create, or change?
- What do users in this role need to be informed of by the system?
- What do users in this role need to inform the system about?

For example, from the point-of-view of the *Student* actor, you may discover that students:

- Enroll in, attend, drop, fail, and pass seminars.
- Need a list of available seminars.
- Need to determine basic information about a seminar, such as its description and its prerequisites.
- Obtain a copy of their transcript, their course schedules, and the fees due.
- Pay fees, pay late charges, receive reimbursements for dropped and cancelled courses, receive grants, and receive student loans.
- Graduate from school or drop out of it.
- Need to be informed of changes in seminars, including room changes, time changes, and even cancellations.

Remaining Agile

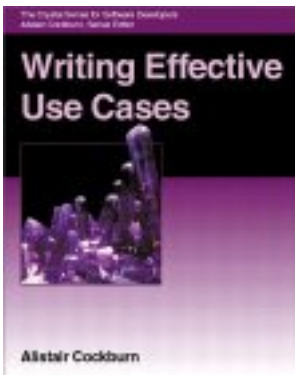
It is very easy for use case modeling to become un-agile. To prevent this from happening you need to focus on creating artifacts that are just barely good enough, they don't need to be perfect. I've seen too many projects go astray because people thought that the requirements had to be worded perfectly. You're not writing the Magna Carta!!!!!! For example, in [Figure 2](#) there are several imperfections, the alternate courses aren't labeled in order (D appears after F and G) and the letters C and E aren't used (they were at some point in the past but then were dropped). The use case isn't perfect yet the world hasn't ended. Yes I could invest time to fix these issues but what would the value be? Nothing. Always remember AM's [Maximize Stakeholder Investment](#) principle and only do things that add value.

Repeat after me: My use cases need to be just good enough. My use cases need to be just good enough. My use cases need to be just good enough. Why does this work? Because in an agile environment you'll quickly move to writing code based on those requirements, you'll discover that you don't fully understand what is required, you'll work closely with your stakeholder to do so, and you'll build

something that meets their actual needs. It's software development, not documentation development.

Notice: This artifact description has been excerpted from [The Object Primer 3rd Edition: Agile Modeling Driven Development with UML 2](#). The book goes into greater detail.

Suggested Reading



This is the best book to read if you want to learn how how write use cases effectively.



This is a very good book describing techniques for working with your stakeholders, including use case modeling.

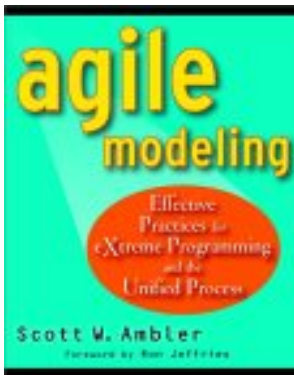
Other artifact
overviews [Use Case Diagrams](#) and [Essential Use Cases](#).

Other Use Case
Writings

- [UML Use Case Diagram Style Guidelines](#)
- [Reuse in Use Case Models](#)

Additional Reading

To learn more about Agile Modeling (AM), consider the book



- [Agile Modeling Home Page](#)
- [Agile Modeling Essays](#)
- [Agile Data Home Page](#)
- [Artifacts for Agile Modeling: The UML and Beyond](#)
- [Modeling Style Home Page](#)

Let Us Help

[Ronin International, Inc.](#) continues to help numerous organizations to learn about and hopefully adopt agile techniques and philosophies. We offer both [consulting](#) and [training](#) offerings. In addition we host several sites - [Agile Modeling](#), [Agile Database Techniques](#), [UML Modeling Style Guidelines](#), [Enterprise Unified Process](#)

[\(EUP\)](#) - that you may find of value.

You might find several of my books to be of interest, including [The Object Primer](#), [Agile Modeling](#), [The Elements of UML Style](#), and [Agile Database Techniques](#).

For more information please contact Michael Vizdos at 866-AT-RONIN (U.S. number) or via e-mail (michael.vizdos@ronin-intl.com).



visits since June 17, 2003.

The Official Agile Modeling (AM) Site



[Modeling Artifacts](#)>Essential Use Cases

Essential Use Case Overview

by [Scott W. Ambler](#), Copyright 2003

An essential use case ([Constantine and Lockwood 1999](#)), sometimes called a business use case, is a simplified, abstract, generalized use case that captures the intentions of a user in a technology and implementation independent manner. A fully documented essential use case is a structured narrative, expressed in the language of the application domain and of users, comprising a simplified, abstract, technology-free and implementation-independent description of one task or interaction. An essential use case is complete, meaningful, and well designed from the point-of-view of users in some role or roles in relation to a system and that embodies the purpose or intentions underlying the interaction.

It isn't quite accurate to say that an essential use case is a business use case in the RUP sense of the term, although they are very close. A business use case is often more focused on the business process and existing technology concerns are often brought into them. Think of them as somewhere in between essential and system use cases, although leaning towards the essential end of the spectrum.

[Figure 1](#) presents an example of a detailed essential use case. Notice how brief and to the point the language is. There isn't a lot of detail because you only need to get the basic idea across. The language of the application domain and of users is used, comprising a simplified, generalized, abstract, technology-free and implementation-independent description of one task or interaction. An essential use case is complete, meaningful, and well designed from the point-of-view of users in some role or roles in relation to a system and that embodies the purpose or intentions underlying the interaction.

Figure 1. Essential use case.

Enroll In Seminar

ID: UC 17

Preconditions:

- The student is enrolled in the university.

Postconditions:

- None

Actor(s)	Response
Student identifies himself	Verifies eligibility to enroll via <i>BR129 Determine Eligibility to Enroll</i> .
	Indicate available seminars
Choose seminar	Validate choice via <i>BR130 Determine Student Eligibility to Enroll in a Seminar</i> .
	Validate schedule fit via <i>BR143 Validate Student Seminar Schedule</i>
	Calculates fees via <i>BR 180 Calculate Student Fees</i> and <i>BR45 Calculate Taxes for Seminar</i> .
	Summarize fees
	Request confirmation
Confirm enrollment	Enroll student in seminar

	Add fees to student bill
	Provide confirmation of enrollment

Essential use cases are typically written in two column format, the column on the left indicates what the actors do and the column on the right the response to their actions. The actor(s) will do something and receive one or more responses to that action. As you can see the flow of the use case is apparent from the spacing of the actions and responses, although you may decide to number the steps to make it more apparent.

The use case refers to [business rules](#) but does not embed the rule in the use case, reflecting AM's practice of [Apply the Right Artifact\(s\)](#). This is a style issue, albeit one that over the years has enabled me to travel light by only recording information in one place (other artifacts will need to refer to business rules as well).

[Constantine and Lockwood \(1999\)](#) use the terms *User Actions* and *System Response* for the headings of these columns although as you can see I use *Actor(s)* and *Response*. The term *Actor(s)* is consistent with common use case terminology and the term *Response* is technology independent – *System Response* may lead some to believe that you must automate everything, which clearly isn't the case. As always, find the style that works best for you.

Notice how I've chosen to indicate a unique identifier for the use case as well as its preconditions and postconditions. These three pieces of information are optional although very useful. I indicate an identifier whenever I need to reference an artifact somewhere else – for example the use case itself references business rules, each of which have a unique identifier. The preconditions, if any, indicate what must be true before this use case is allowed to run. The postconditions, if any, indicate what will be true once the use case finishes successfully.

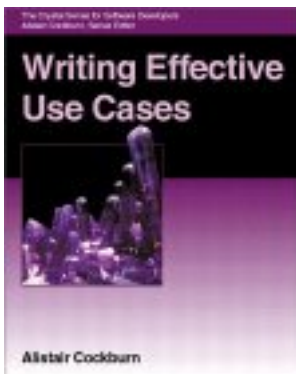
A significant advantage of essential use cases is that they enable you to stand back and ask fundamental questions like "what's really going on" and "what do we really need to do" without letting implementation decisions get in the way. These questions often lead to critical realizations that allow you to rethink, or reengineer if you prefer that term, aspects of the overall business process.

When you are essential use case modeling you can be technology independent but you will never be perfectly “system independent”. You will always be bound by the scope of your effort, but the vision/goals of your project. As a result the perceived system that you’re building, be it manual, automomated, or a hybrid of the two, will always affect your how you write your use cases.

Why essential use cases? [Traditional/system use cases](#) typically contain too many built-in assumptions, often hidden or implicit, about the underlying technology implementation and the user interface yet to be designed. This is a good feature during your analysis and design efforts, but not so good for your requirement efforts. An essential use case, on the other hand, is based on the purpose or intentions of a user, rather than on the concrete steps or mechanisms by which the purpose or intention might be carried out.

Notice: This artifact description has been excerpted from [The Object Primer 3rd Edition: Agile Modeling Driven Development with UML 2](#). The book goes into greater detail.

Suggested Reading



This is the best book to read if you want to learn how how write use cases effectively.



This is a very good book describing techniques for working with your stakeholders, including use case modeling.

Other artifact
overviews

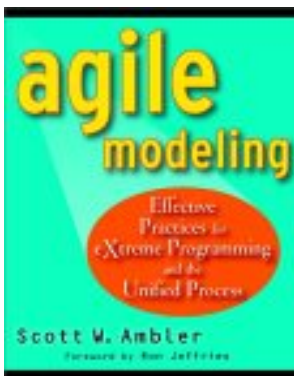
[Use Case Diagrams](#) and [System Use Cases](#).

Other Use Case
Writings

- [UML Use Case Diagram Style Guidelines](#)
- [Reuse in Use Case Models](#)

Additional Reading

To learn more about Agile Modeling (AM), consider the book



- [Agile Modeling Home Page](#)
- [Agile Modeling Essays](#)
- [Agile Data Home Page](#)
- [Artifacts for Agile Modeling: The UML and Beyond](#)
- [Modeling Style Home Page](#)

Translations

- [Japanese](#)

Let Us Help

[Ronin International, Inc.](#) continues to help numerous organizations to learn about and hopefully adopt agile techniques and philosophies. We offer both [consulting](#) and [training](#) offerings. In addition we host several sites - [Agile Modeling](#), [Agile Database Techniques](#), [UML Modeling Style Guidelines](#), [Enterprise Unified Process \(EUP\)](#) - that you may find of value.

You might find several of my books to be of interest, including [The Object Primer](#), [Agile Modeling](#), [The Elements of UML Style](#), and [Agile Database Techniques](#).

For more information please contact Michael Vizados at 866-AT-RONIN (U.S. number) or via e-mail (michael.vizados@ronin-intl.com).



visits since July 17, 2003.

The Official Agile Modeling (AM) Site



Essay

Reuse in Use-Case Models: <<extend>>, <<include>>, and Inheritance

by [Scott W. Ambler](#), Copyright 2001-2002

Note: This essay is excerpted from Chapter 6 of the book [The Object Primer 2/e](#).

One of your goals during analysis is to identify potential opportunities for reuse, a goal you can work toward as you are developing your use-case model. Potential reuse can be model through four generalization relationships supported by UML use-case models:

- [Extend dependencies between use cases](#)
- [Include dependencies between use cases](#)
- [Inheritance between use cases](#)
- [Inheritance between actors](#)

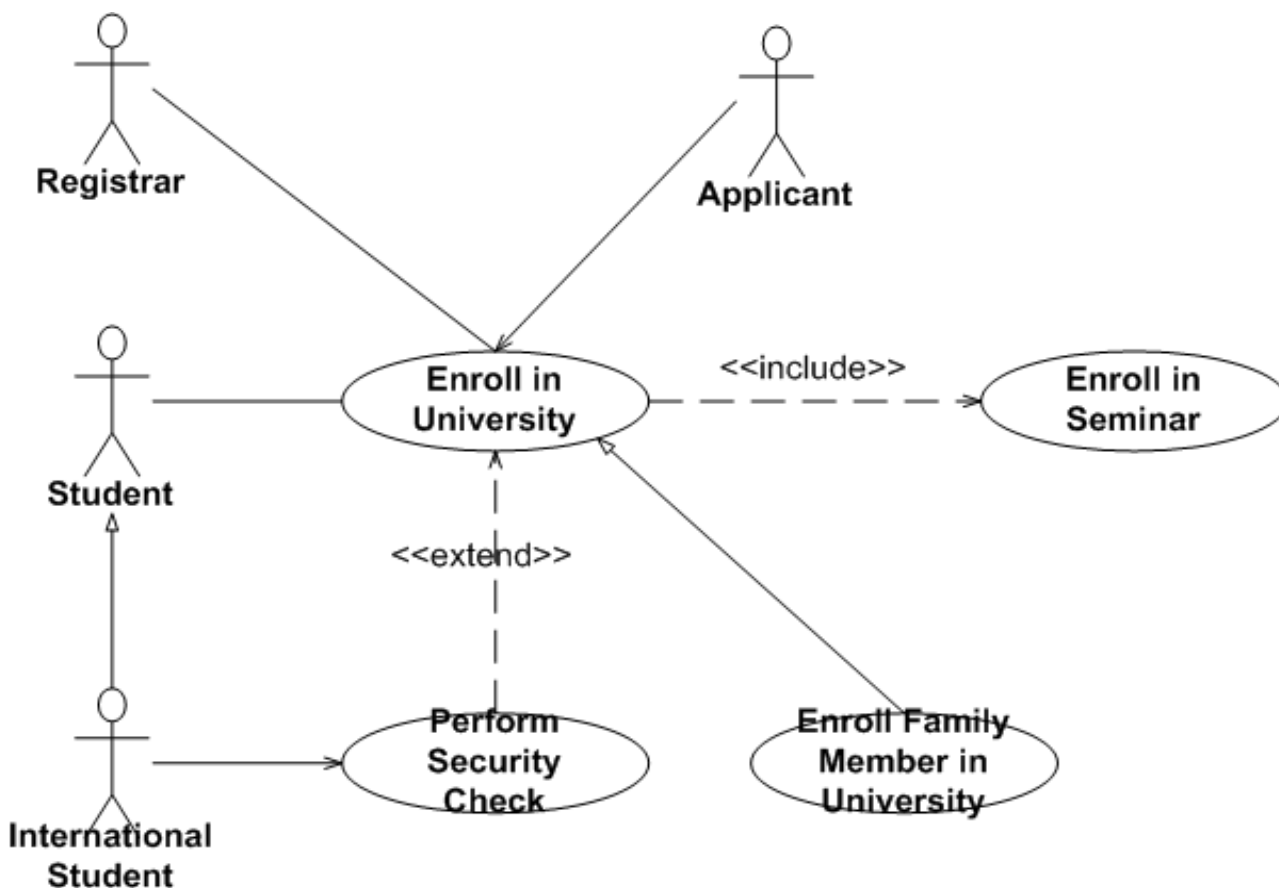
Extend Dependencies Between Use Cases

An extend dependency, formerly called an extends relationship in UML v1.2 and earlier, is a generalization relationship where an extending use case continues the behavior of a base use case. The extending use case accomplishes this by conceptually inserting additional action sequences into the base use-case sequence. This allows an extending use case to continue the activity sequence of a base use case when the appropriate extension point is reached in the base use case and the extension condition is fulfilled. When the extending use case activity sequence is completed, the base use case continues. In [Figure 1](#), you see the use

case “Enroll International Student in University” extends the use case “Enroll in University,” the notation for doing so is simply a normal use-case dependency with the stereotype of <<extend>>. In this case, “Enroll in University” is the base use case and “Enroll International Student in University” is the extending use case.

An extending use case is, effectively, an alternate course of the base use case. In fact, a good rule of thumb is you should introduce an extending use case whenever the logic for an alternate course of action is at a complexity level similar to that of your basic course of action. I also like to introduce an extending use case whenever I need an alternate course for an alternate course; in this case, the extending use case would encapsulate both alternate courses. Many use-case modelers avoid the use of extend dependencies as this technique has a tendency to make use-case diagrams difficult to understand. My preference is to use extend dependencies sparingly. Note, the extending use case—in this case “Enroll International Student in University”—would list “UC33 Enroll in University,” the base use case, in its “Extends” list.

Figure 1 . The opportunities for reuse in use-case models.



Just as you indicate the point at which the logic of an alternate course replaces the

logic of a portion of the basic course of action for a use case, you need to be able to do the same thing for an extending use case. This is accomplished through the use of an extension point, which is simply a marker in the logic of a base use case indicating where extension is allowed. [Figure 3](#) presents an example of how an extension point would be indicated in the basic course of action of the “Enroll In University” use case of [Figure 2](#). Notice how the identifier and the name of the use case is indicated. If several use cases extended this one from the same point, then each one would need to be listed. A condition statement, such as “Condition: Enrollee is an international student,” could have been indicated immediately following the name of the use case but, in this example, it was fairly obvious what was happening.

Figure 2. The Enroll in University Use Case (narrative style).

Name: Enroll in Seminar

Identifier: UC 17

Description:

Enroll an existing student in a seminar for which she is eligible.

Preconditions:

The Student is registered at the University.

Postconditions:

The Student will be enrolled in the course she wants if she is eligible and room is

available.

Extends:

-

Includes:

-

Inherits From:

-

Basic Course of Action:

1. The student wants to enroll in a seminar.
2. The student inputs his name and student number into the system via "UI23 Security Login Screen."
3. The system verifies the student is eligible to enroll in seminars at the university, according to business rule "BR129 Determine Eligibility to Enroll."
4. The system displays "UI32 Seminar Selection Screen," which indicates the list of available seminars.
5. The student indicates the seminar in which he wants to enroll.
6. The system validates the student is eligible to enroll in the seminar, according to the business rule "BR130 Determine Student Eligibility to Enroll in a Seminar."
7. The system validates the seminar fits into the existing schedule of the student, according to the business rule "BR143 Validate Student Seminar Schedule."
8. The system calculates the fees for the seminar based on the fee published in the course catalog, applicable student fees, and applicable taxes. Apply

business rules “BR 180 Calculate Student Fees” and “BR45 Calculate Taxes for Seminar.”

9. The system displays the fees via “UI33 Display Seminar Fees Screen.”
10. The system asks the student whether he still wants to enroll in the seminar.
11. The student indicates he wants to enroll in the seminar.
12. The system enrolls the student in the seminar.
13. The system informs the student the enrollment was successful via “UI88 Seminar Enrollment Summary Screen.”
14. The system bills the student for the seminar, according to business rule ‘BR100 Bill Student for Seminar.’
15. The system asks the student if he wants a printed statement of the enrollment.
16. The student indicates he wants a printed statement.
17. The system prints the enrollment statement “UI89 Enrollment Summary Report.”
18. The use case ends when the student takes the printed statement.

Alternate Course A: The Student is Not Eligible to Enroll in Seminars

- A.3. The system determines the student is not eligible to enroll in seminars.
- A.4. The system informs the student she is not eligible to enroll.
- A.5. The use case ends.

Alternate Course B: The Student Does Not Have the Prerequisites

- B.6. The system determines the student is not eligible to enroll in the seminar she has chosen.
- B.7. The system informs the student she does not have the prerequisites.
- B.8. The system informs the student of the prerequisites she needs.
- B.9. The use case continues at Step 4 in the basic course of action.

Alternate Course C: The Student Decides Not to Enroll in an Available Seminar

C.4. The student views the list of seminars and doesn't see one in which he wants to enroll.

C.5. The use case ends.

Figure 3 . Documenting an extension point within a use case.

4. The system displays "UI43 Student Information Entry". [Extension Point: *UC34 Enroll International Student In University*].

5. The student...

Include Dependencies Between Use Cases

A second way to indicate potential reuse within use-case models exists in the form of include dependencies. An include dependency, formerly known as a uses relationship in UML v1.2 and earlier, is a generalization relationship denoting the inclusion of the behavior described by another use case. The best way to think of an include dependency is that it is the invocation of a use case by another one. In [Figure 1](#) , you see the use case "Enroll in University" includes the use case "Enroll in Seminar," the notation for doing so is simply a normal use-case dependency with the stereotype of <<include>>. [Figure 4](#) presents an example of how you would indicate where the use case is included in the logic of the including use case. Similar to calling a function or invoking an operation within source code, isn't it?

Figure 4 . Indicating the inclusion of a use case.

8. The student indicates the seminar(s) she wants to take via the use case *UC 17 Enroll in Seminar*.
9. The student...

You use include dependencies whenever one use case needs the behavior of another. Introducing a new use case that encapsulates similar logic that occurs in several use cases is quite common. For example, you may discover several use cases need the behavior to search for and then update information about students, indicating the potential need for an “Update Student Record” use case included by the other use cases.

As you would expect, the use case “Enroll in University” should list “UC17 Enroll in Seminar” in its “Includes” list. Why should you bother maintaining an “Includes” and an “Extends” list in your use cases? The answer is simple: your use cases should stand on their own, you shouldn’t expect people to have your use-case diagram in front of them. Yes, it would be nice if everyone has access to the use-case diagram because it also contains this information, but the reality is that sometimes you use different tools to document each part of your model. For example, your diagrams could be drawn using a drawing package and your use cases documented in a word processor. Some of your project stakeholders may have access to the word processor you are using, but not the drawing package. The main disadvantage of this approach is you need to maintain these two lists in parallel with the diagram, the danger being they may become unsynchronized.

Inheritance Between Use Cases

Use cases can inherit from other use cases, offering a third opportunity to indicate potential reuse. [Figure 1](#) depicts an example of this, showing that “Enroll Family Member in University” inherits from the “Enroll In University” use case. Inheritance between use cases is not as common as either the use of extend or include dependencies, but it is still possible. The inheriting use case would completely replace one or more of the courses of action of the inherited use case. In this case, the basic course of action is completely rewritten to reflect that new business rules are applied when the family member of a professor is enrolling at the university. Family members are allowed to enroll in the school, regardless of the marks they

earned in high school, they don't have to pay any enrollment fees, and they are given top priority for enrollment in the university.

Inheritance between use cases should be applied whenever a single condition, in this case, the student is a family member of a professor, would result in the definition of several alternate courses. Without the option to define an inheriting use case, you need to introduce an alternate course to rework the check of the student's high-school marks, the charging of enrollment fees, and for prioritization of who is allowed to enroll in the given semester.

The inheriting use case is much simpler than the use case from which it inherits. It should have a name, description, and identifier, and it should also indicate from which use case it inherits in the "Inherits From" section. This includes any section that is replaced, particularly the pre-conditions and post-conditions as well as any courses of action. If something is not replaced, then leave that section blank, assuming it is inherited from the parent use case (you might want to put text, such as "see parent use case," in the section).

Inheritance Between Actors

The fourth opportunity for indicating potential reuse within use-case models occurs between actors: an actor on a use-case diagram can inherit from another actor. An example of this is shown in [Figure 1](#), the "International Student" actor inherits from "Student." An international student is a student, the only difference being is he or she is subject to different rules and policies (the international student pays more in tuition and is restricted from taking several seminars because of political reasons). The standard UML notation for inheritance, the open-headed arrow, is used and the advice presented about the appropriate use of inheritance still applies: it should make sense to say the inheriting actor is or is like the inherited actor.

References and Suggested Reading

See the [references page](#).

[The Object Primer 2/e](#)

[Agile Modeling Home Page](#)

[Agile Modeling Essays](#)

[Use Case Modeling Style Guidelines](#)